Math for Data Science - Lab 8

Course: Math for Data Science

This lab is a hands-on tour of the linear algebra you need for the rest of the course: matrix addition and multiplication, transposes, norms, determinants, linear independence, inverses, span, and the singular value decomposition. For most exercises you should **compute the answer by hand first**, then check it with NumPy.

Questions marked *[optional]* can be skipped if you are short of time.

Setup

Run the cell below once to import everything we need.

```
import numpy as np
```

Using R instead of Python (optional)

The lab is written in Python, but you can write your answers in R inside this same notebook. This needs R 4.5 or newer, and Jupyter started with rpy2 added: `uv run --with rpy2 jupyter lab`. The `README.md` of the course folder has the details and what to do when it does not work.

If you want to use R instead of Python, uncomment and run the following cell:

```
# %load_ext rpy2.ipython
```

Then, in any subsequent code cell, add `%%R` as the first line to run that cell as R:

```
%%R
x <- rbinom(1000, size = 10, prob = 0.3)
hist(x)
```

Keep running the given Python cells (imports, data loading) as they are. The hints name Python functions, such as `rng.binomial`; you will need to find the R equivalent (`rbinom`) yourself.

Matrices

A matrix $\mathbf{A}$ with m rows and n columns is

$$\text{Matrix } \mathbf{A} = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{bmatrix}$$

An element of this matrix is $a_{i,j}$ where i is the row and j is the column. This is the same as indexing a NumPy array: `A[i, j]`.

> **Watch out:** Python indexes from **0**, so the element mathematicians call $a_{1,1}$ is `A[0, 0]` in code.

Exercise 1: Matrix Addition

In NumPy, matrices are made with `np.array` from a list of rows - so arrays are filled **by row** by default:

```
A = np.array([[1, 4, 7],
              [2, 5, 8],
              [3, 6, 9]])

B = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

print("A =", A, sep="\n")
print("\nB =", B, sep="\n")
```

Notice that `A` holds the numbers 1–9 filled **by column** and `B` holds them filled **by row**. A quick way to build the column-filled version is to reshape and transpose, or to use the `order="F"` (Fortran, i.e. column-major) argument:

```
np.arange(1, 10).reshape(3, 3, order="F")   # same as A
np.arange(1, 10).reshape(3, 3)              # same as B
```

So

$$\mathbf{A} = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

What is $\mathbf{A} + \mathbf{B}$? Compute by hand and then with NumPy. We'll call the resulting matrix $\mathbf{C}$.

here your solution

```
# here your code
```

[optional] Now consider

$$\mathbf{D} = \begin{bmatrix} 4 & 12 \\ 8 & 16 \end{bmatrix}$$

What about $\mathbf{A} + \mathbf{B} + \mathbf{D}$?

```
D = np.array([[4, 12],
              [8, 16]])
```

here your solution

```
# here your code
```

Exercise 2: Matrix Multiplication

Multiplying a matrix by a scalar

Multiplying a matrix by a scalar (single value) means multiplying each element of the matrix by the scalar. For example, let's try $2 \times \mathbf{A}$.

here your solution

```
# here your code
```

Warning: Broadcasting in NumPy

NumPy will silently *broadcast* a smaller array against a larger one when the shapes are compatible. For example, NumPy is perfectly happy to run:

```
np.array([1, 2, 3]) * A
```

No error, no warning - but this is **not** a linear algebra operation. NumPy stretched the vector `[1, 2, 3]` across the rows, so it multiplied *column* j of $\mathbf{A}$ by element j of the vector:

$$\begin{bmatrix} 1 \cdot 1 & 2 \cdot 4 & 3 \cdot 7 \\ 1 \cdot 2 & 2 \cdot 5 & 3 \cdot 8 \\ 1 \cdot 3 & 2 \cdot 6 & 3 \cdot 9 \end{bmatrix} = \begin{bmatrix} 1 & 8 & 21 \\ 2 & 10 & 24 \\ 3 & 12 & 27 \end{bmatrix}$$

The equivalent explicit operation is multiplying elementwise by a full-size matrix built by repeating the vector down the rows:

```
np.tile(np.array([1, 2, 3]), (3, 1))
```

This silent broadcasting is a common source of bugs: if you meant matrix multiplication and typed `*`, you will often get an answer rather than an error. **When you mean matrix multiplication, use `@`.**

> Broadcasting only applies when the shapes are compatible. Try `np.array([1, 2]) * A` and NumPy *will* raise an error, because length 2 cannot be stretched to match 3 columns.

Multiplying a matrix by a vector or matrix

To multiply matrices, they must be **conformable**: $m \times n$ and $n \times p$, for an output of $m \times p$. In other words, the number of columns in the first matrix must equal the number of rows in the second matrix. The shared dimension n is what allows the dot product operation to occur between each row of the first matrix and each column of the second matrix.

Therefore **order matters** in matrix multiplication.

Recall the example from lecture:

$$\begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \end{bmatrix} \times \begin{bmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \\ b_{3,1} & b_{3,2} \end{bmatrix} = \begin{bmatrix} a_{1,1}b_{1,1} + a_{1,2}b_{2,1} + a_{1,3}b_{3,1} & a_{1,1}b_{1,2} + a_{1,2}b_{2,2} + a_{1,3}b_{3,2} \\ a_{2,1}b_{1,1} + a_{2,2}b_{2,1} + a_{2,3}b_{3,1} & a_{2,1}b_{1,2} + a_{2,2}b_{2,2} + a_{2,3}b_{3,2} \end{bmatrix}$$

Multiplying a matrix with 2 rows and 3 columns by a matrix with 3 rows and 2 columns yields a matrix with 2 rows and 2 columns. Reversing the order would yield a matrix with 3 rows and 3 columns.

[optional] Let's first multiply two vectors, $\mathbf{a}$ and $\mathbf{b}$. Recall that a vector can be treated as a matrix with either 1 column or 1 row (1 row is typically denoted as the transpose of a vector). Calculate $\mathbf{a}^T\mathbf{b}$ where

$$\mathbf{a}^T = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \quad \text{and} \quad \mathbf{b} = \begin{bmatrix} 3 \\ 6 \\ 9 \end{bmatrix}$$

```
a = np.array([[1, 2, 3]])       # 1 x 3 (a row vector, i.e. a transposed)
b = np.array([[3], [6], [9]])   # 3 x 1 (a column vector)
```

here your solution

```
# here your code
```

Let's try multiplying two 2×2 matrices by hand, first $\mathbf{A} \times \mathbf{B}$, then $\mathbf{B} \times \mathbf{A}$, where

$$\mathbf{A} = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}$$

The NumPy operator for matrix multiplication is `@`.

```
A = np.array([[1, 3],
              [2, 4]])

B = np.array([[0, 0],
              [1, 1]])
```

here your solution

```
# here your code
```

Using * will do **elementwise** multiplication (multiply each element by the corresponding element in the other matrix by row and column index). Compare:

```
A * B
```

[optional] Now let's try multiplication with the matrices

$$\mathbf{A} = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 1 & 2 & 5 \\ 1 & 3 & 8 \end{bmatrix}$$

First try $\mathbf{A} \times \mathbf{B}$, then $\mathbf{B} \times \mathbf{A}$.

```
A = np.array([[1, 4],
              [2, 5],
              [3, 6]])

B = np.array([[1, 2, 5],
              [1, 3, 8]])
```

here your solution

```
# here your code
```

[optional] Now let's try matrix multiplication with

$$\mathbf{A} = \begin{bmatrix} 5 & 15 \\ 10 & 20 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

What is $\mathbf{A} \times \mathbf{B}$? How about $\mathbf{B} \times \mathbf{A}$?

```
A = np.array([[5, 15],
              [10, 20]])

B = np.array([[1, 3, 5],
              [2, 4, 6]])
```

here your solution

```
# here your code
```

Exercise 3: Transpose of a Vector or Matrix

You can take the transpose of a matrix in NumPy with `.T`. For example, the transpose of

$$\mathbf{A} = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

is

$$\mathbf{A}^T = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

```
A = np.array([[1, 3, 5],
              [2, 4, 6]])
A.T
```

[optional] Using the matrices

$$\mathbf{A} = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$$

and the vectors

$$\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \quad \text{and} \quad \mathbf{b} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

prove to yourself that

$$(\mathbf{A}^T)^T = \mathbf{A}$$

$$(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T$$

$$(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T$$

$$\mathbf{a}^T\mathbf{b} = \mathbf{b}^T\mathbf{a}$$

```
A = np.array([[1, 3],
              [2, 4]])

B = np.array([[1, 1],
              [0, 0]])

a = np.array([1, 2, 3, 4])
b = np.array([1, 0, 1, 0])
```

```
# here your code
```

Exercise 4: Norm of a Vector

The norm of a vector is a measure of its magnitude, or distance from 0. The L_1 and L_2 norms are the two most common.

$$L_1 \text{ Norm } \| \mathbf{x} \|_1 = \sum_i |x_i|$$

$$L_2 \text{ Norm } \| \mathbf{x} \|_2 = \sqrt{\sum_i x_i^2}$$

Find the L_1 and L_2 norms of the vector

$$\mathbf{v} = \begin{bmatrix} 1 \\ -2 \\ 0 \\ 4 \end{bmatrix}$$

```
v = np.array([1, -2, 0, 4])
```

here your solution

```
# here your code
```

Exercise 5: Determinant of a Matrix

The determinant of a square matrix, denoted $\det(\mathbf{A})$ or $|\mathbf{A}|$, reduces the elements of the matrix to a single number.

Let's take the determinant of

$$\mathbf{A} = \begin{bmatrix} 1 & 5 \\ 3 & 7 \end{bmatrix}$$

```
A = np.array([[1, 5],
              [3, 7]])
```

here your solution

```
# here your code
```

Exercise 6: Linear Independence

Recall that a set of vectors is **linearly dependent** if at least one vector in the set can be written as a linear combination of the other vectors in the set. A set of vectors is **linearly independent** if it is not linearly dependent.

Using mathematical notation, the vectors are linearly dependent if

$$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + ... + c_n\mathbf{v}_n = \mathbf{0}$$

for some $c_i \neq 0$.

True or False? The vectors

$$\mathbf{v}_1 = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 4 \\ 6 \\ 2 \end{bmatrix}$$

are linearly independent.

here your solution

```
# here your code
```

Exercise 7: Inverse of a Matrix

You will not be asked to invert a matrix by hand, but you should know what a matrix inverse is and how to use it:

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$$

Not all matrices are invertible. A matrix $\mathbf{A}$ is invertible (i.e. $\mathbf{A}^{-1}$ exists) *if and only if* $\det(\mathbf{A}) \neq 0$. This implicitly requires that the matrix is square (to have a determinant) and that its columns are linearly independent ("full rank").

You can invert a matrix with `np.linalg.inv`.

Question. For $\mathbf{B} = \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix}$, $\mathbf{B}^{-1}$ is:

a. $\begin{bmatrix} -3 & 0 \\ 0 & -3 \end{bmatrix}$

b. $\begin{bmatrix} \frac{1}{3} & 0 \\ 0 & \frac{1}{3} \end{bmatrix}$

c. $\begin{bmatrix} 0 & \frac{1}{3} \\ \frac{1}{3} & 0 \end{bmatrix}$

d. $\begin{bmatrix} -\frac{1}{3} & 0 \\ 0 & -\frac{1}{3} \end{bmatrix}$

here your solution

```
# here your code
```

[optional] **True or False?** The matrix $\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 2 & 0 \end{bmatrix}$ is invertible.

here your solution

```
# here your code
```

Span of a Set of Vectors

The **span** of a set of vectors S is all linear combinations of the vectors in the set.

If V is a vector space and S a set of vectors in V, then S is a **spanning set** for V if $V = \text{Span}(S)$.

Exercise 8: Singular Value Decomposition

Perform a step-by-step singular value decomposition on the matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 2 \\ 2 & 1 & 0 \end{bmatrix}$$

```
A = np.array([[1, 0, 2],
              [2, 1, 0]])
A
```

1. Write down $\mathbf{A}^T$

```
# here your code
```

2. Compute $\mathbf{A}\mathbf{A}^T$ and $\mathbf{A}^T\mathbf{A}$

here your solution

```
# here your code
```

3. Find the eigenvalues of $\mathbf{A}\mathbf{A}^T$ and $\mathbf{A}^T\mathbf{A}$

You don't neccesarily need it but in case you do, remember that the roots of a second-order polynomial, or quadratic equation, in the form $ax^2 + bx + c = 0$ is given by

$$x = \frac{-b + \pm\sqrt{b^2 - 4ac}}{2a}$$

> **Tip:** use `np.linalg.eigh` for symmetric matrices - it is more accurate than the general `np.linalg.eig` and returns real eigenvalues in **ascending** order.

here your solution

```
# here your code
```

4. Write down Σ, sorted in decreasing order of the singular values

here your solution

```
# here your code
```

5. How many linearly independent columns are needed to form a basis for the column space of A?

6. How many linearly independent rows are needed to form a basis for the row space of A?

[optional]

here your solution

```
# here your code
```

Reconstructing the Original Matrix

[optional]

Now reconstruct $\mathbf{A}$ from $\mathbf{U}$, $\mathbf{\Sigma}$, and $\mathbf{V}^T$. Watch out for a subtlety.

```
# here your code
```

here your solution

```
# here your code
```